นายรติพงษ์ ครองระวะ 664245045 66/44

1.สร้าง Node

Property keys

club_id | interest_name | name | room | student_id | year

```
CREATE
  (s1:Student {student_id:'S001', name:'Alice',   year:2}),
  (s2:Student {student_id:'S002', name:'Bob',     year:2}),
  (s3:Student {student_id:'S003', name:'Charlie', year:3}),
  (s4:Student {student_id:'S004', name:'Diana',   year:2}),
  (s5:Student {student_id:'S005', name:'Ethan',   year:1}),

  (c1:Club {club_id:'C01', name:'Robotics Club',    room:'ICT-201'}),
  (c2:Club {club_id:'C02', name:'Music Club',       room:'ART-105'}),
  (c3:Club {club_id:'C03', name:'Photography Club', room:'LIB-302'}),

  (i1:Interest {interest_name:'Programming'}),
  (i2:Interest {interest_name:'AI'}),
  (i3:Interest {interest_name:'Music'}),
  (i4:Interest {interest_name:'Photography'});
```

ประเภท	รหัส	ชื่อ	ข้อมูลเพิ่มเติม
Student	S001	Alice	year 2 (เป็นนักเรียนเป้าหมายของระบบแนะนำ)
Student	S002	Bob	year 2
Student	S003	Charlie	year 3
Student	S004	Diana	year 2
Student	S005	Ethan	year 1
Club	C01	Robotics Club	ห้อง ICT-201
Club	C02	Music Club	ห้อง ART-105
Club	C03	Photography Club	ห้อง LIB-302
Interest	-	Programming, AI, Music, Photography	รวม 4 หัวข้อ

สร้างข้อมูลตั้งต้นตามที่โจทย์กำหนด คือ Student อย่างน้อย 5 คน และ Club 3 ชมรม พร้อมเพิ่ม Interest อีก 4 หัวข้อเพื่อรองรับการให้คะแนนจากความสนใจ การเขียน CREATE ทั้งหมดในคำสั่งเดียวทำให้ Neo4j ทำงานเป็นหนึ่ง transaction ซึ่งเร็วกว่าและปลอดภัยกว่าการแยกหลายคำสั่ง

2.สร้าง Relationship

Relationships (18)

* FOCUSES_ON FRIEND_OF
INTERESTED_IN MEMBER_OF

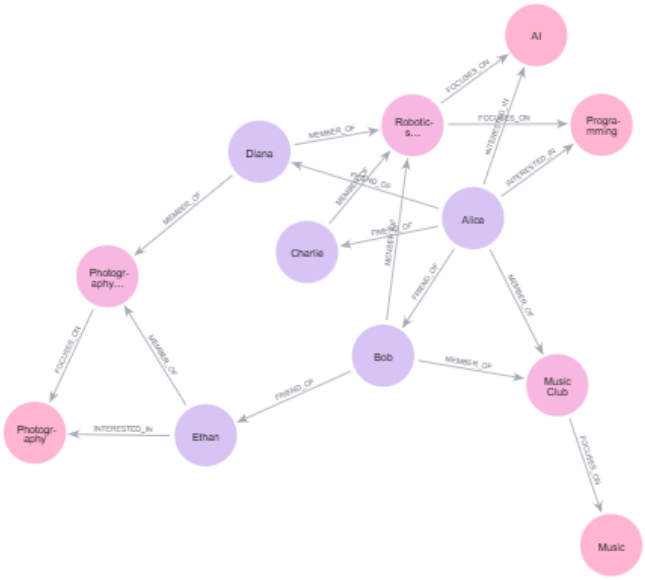
```
// FRIEND_OF : สร้างทิศเดียว แล้วตอน MATCH ใช้แบบไร้ทิศทาง
MATCH (a:Student {student_id:'S001'}), (b:Student {student_id:'S002'}) CREATE (a)-
[:FRIEND_OF]->(b);
MATCH (a:Student {student_id:'S001'}), (b:Student {student_id:'S003'}) CREATE (a)-
[:FRIEND_OF]->(b);
MATCH (a:Student {student_id:'S001'}), (b:Student {student_id:'S004'}) CREATE (a)-
[:FRIEND_OF]->(b);
MATCH (a:Student {student_id:'S002'}), (b:Student {student_id:'S005'}) CREATE (a)-
[:FRIEND_OF]->(b);

// MEMBER_OF : S001 อยู่ C02 อยู่แล้ว (ต้องถูกตัดออกจากผลลัพธ์)
MATCH (s:Student {student_id:'S001'}), (c:Club {club_id:'C02'}) CREATE (s)-
[:MEMBER_OF]->(c);
MATCH (s:Student {student_id:'S002'}), (c:Club {club_id:'C01'}) CREATE (s)-
[:MEMBER_OF]->(c);
MATCH (s:Student {student_id:'S002'}), (c:Club {club_id:'C02'}) CREATE (s)-
[:MEMBER_OF]->(c);
MATCH (s:Student {student_id:'S003'}), (c:Club {club_id:'C01'}) CREATE (s)-
[:MEMBER_OF]->(c);
MATCH (s:Student {student_id:'S004'}), (c:Club {club_id:'C01'}) CREATE (s)-
[:MEMBER_OF]->(c);
MATCH (s:Student {student_id:'S004'}), (c:Club {club_id:'C03'}) CREATE (s)-
[:MEMBER_OF]->(c);
MATCH (s:Student {student_id:'S005'}), (c:Club {club_id:'C03'}) CREATE (s)-
[:MEMBER_OF]->(c);

// INTERESTED_IN
MATCH (s:Student {student_id:'S001'}), (i:Interest {interest_name:'Programming'})
CREATE (s)-[:INTERESTED_IN]->(i);
MATCH (s:Student {student_id:'S001'}), (i:Interest {interest_name:'AI'})
CREATE (s)-[:INTERESTED_IN]->(i);
MATCH (s:Student {student_id:'S005'}), (i:Interest {interest_name:'Photography'})
CREATE (s)-[:INTERESTED_IN]->(i);

// FOCUSES_ON
MATCH (c:Club {club_id:'C01'}), (i:Interest {interest_name:'Programming'}) CREATE
(c)-[:FOCUSES_ON]->(i);
MATCH (c:Club {club_id:'C01'}), (i:Interest {interest_name:'AI'})          CREATE
(c)-[:FOCUSES_ON]->(i);
MATCH (c:Club {club_id:'C02'}), (i:Interest {interest_name:'Music'})       CREATE
(c)-[:FOCUSES_ON]->(i);
MATCH (c:Club {club_id:'C03'}), (i:Interest {interest_name:'Photography'}) CREATE
(c)-[:FOCUSES_ON]->(i);
```

นักเรียน	เพื่อน	ชมรมที่เป็นสมาชิก	ความสนใจ
Alice (S001)	Bob, Charlie, Diana	Music Club	Programming, AI
Bob (S002)	Alice, Ethan	Robotics Club, Music Club	-
Charlie (S003)	Alice	Robotics Club	-
Diana (S004)	Alice	Robotics Club, Photography Club	-
Ethan (S005)	Bob	Photography Club	Photography



ขั้นนี้คือหัวใจของ Graph Database เพราะคุณค่าของกราฟอยู่ที่เส้นเชื่อม ไม่ใช่ที่ตัว Node จุดที่ออกแบบไว้อย่างตั้งใจคือ ให้ Alice (S001) เป็นสมาชิก Music Club อยู่แล้ว เพื่อให้เห็นชัดในขั้นถัดไปว่าระบบสามารถตัดชมรมที่เป็นสมาชิกอยู่แล้วออกจากผลลัพธ์ได้จริง

3.MATCH + WHERE + RETURN ดูชมรมของเพื่อน S001

	friend_name	club_id	club_name
1	"Bob"	"C01"	"Robotics Club"
2	"Charlie"	"C01"	"Robotics Club"
3	"Diana"	"C01"	"Robotics Club"
4	"Bob"	"C02"	"Music Club"
5	"Diana"	"C03"	"Photography Club"

```
MATCH (me:Student {student_id:'S001'})-[:FRIEND_OF]-(friend:Student)-[:MEMBER_OF]-
>(club:Club)
WHERE me <> friend
RETURN friend.name  AS friend_name,
       club.club_id AS club_id,
       club.name    AS club_name
ORDER BY club_id;
```

ในขั้นนี้ Music Club ยังปรากฏอยู่ เพราะเรายังไม่ได้ใส่เงื่อนไขตัดชมรมที่ Alice เป็นสมาชิกอยู่แล้ว

ขั้นนี้เป็นการเดินตาม Pattern ที่โจทย์กำหนดแบบตรงไปตรงมา เพื่อดูว่าเพื่อนของ Alice อยู่ชมรมใดบ้าง สังเกตว่าส่วน FRIEND_OF เขียนแบบไม่มีลูกศร เพราะข้อมูลถูกสร้างไว้ทิศเดียว หากใส่ลูกศรจะได้ผลลัพธ์ไม่ครบ ส่วนเงื่อนไข WHERE me <> friend ใส่ไว้กันกรณีมีข้อมูลที่นักเรียนเป็นเพื่อนกับตัวเอง

4.Traversal ให้คะแนนด้วย count(DISTINCT friend)

	club_id	club_name	friend_score	interest_score	total_score	friend_list
1	"C01"	"Robotics Club"	3	2	8	["Bob", "Charlie", "Diana"]
2	"C03"	"Photography Club"	1	0	2	["Diana"]

```
MATCH (me:Student {student_id:'S001'})-[:FRIEND_OF]-(friend:Student)-[:MEMBER_OF]-
>(club:Club)
WHERE NOT (me)-[:MEMBER_OF]->(club)
WITH me, club, count(DISTINCT friend) AS friend_score,
     collect(DISTINCT friend.name) AS friend_list
OPTIONAL MATCH (me)-[:INTERESTED_IN]->(i:Interest)<-[:FOCUSES_ON]-(club)
WITH club, friend_score, friend_list, count(DISTINCT i) AS interest_score
RETURN club.club_id AS club_id,
       club.name    AS club_name,
       friend_score,
       interest_score,
       friend_score * 2 + interest_score AS total_score,
       friend_list
ORDER BY total_score DESC, club_id;
```

Music Club (C02) หายไปจากผลลัพธ์แล้ว เพราะถูกตัดด้วยเงื่อนไข WHERE NOT ซึ่งพิสูจน์ว่าระบบทำงานตามที่โจทย์กำหนดได้ถูกต้อง

ขั้นนี้คือการแปลงเส้นทางในกราฟให้กลายเป็นคะแนน มีสามจุดสำคัญ

WHERE NOT (me)-[:MEMBER_OF]->(club) ใช้ pattern เป็นเงื่อนไขทางตรรกะ เพื่อตัดชมรมที่ Alice เป็นสมาชิกอยู่แล้วออก ซึ่งเป็นสิ่งที่โจทย์กำหนดไว้ชัดเจน

count(DISTINCT friend) ต้องใส่ DISTINCT เสมอ เพราะหากเพื่อนคนเดียวกันเดินไปถึงชมรมเดียวกันได้หลายเส้นทาง จะถูกนับซ้ำจนคะแนนผิดเพี้ยน

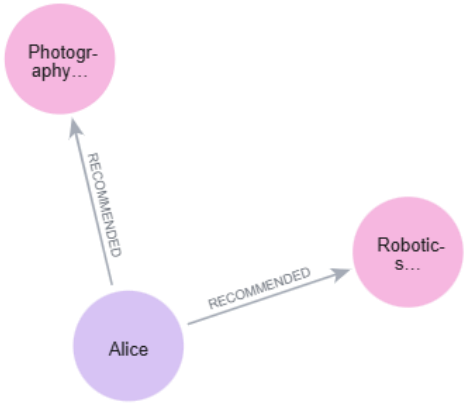
OPTIONAL MATCH ใช้กับส่วนความสนใจ เพราะถ้าเขียนด้วย MATCH ธรรมดา ชมรมที่ไม่มี Interest ตรงกันเลยจะหายไปจากผลลัพธ์ทั้งแถว แต่ OPTIONAL MATCH จะคืนค่าเป็น null แล้วนับได้ 0 แทน

สูตรคะแนนที่ใช้คือ total_score = (จำนวนเพื่อนในชมรม × 2) + (จำนวนความสนใจที่ตรงกัน) โดยให้น้ำหนักเพื่อนมากกว่า เพราะปัจจัยทางสังคมมีผลต่อการตัดสินใจเข้าชมรมของนักเรียนสูงกว่า

5.สร้าง Relationship :RECOMMENDED เก็บผลลัพธ์กลับเข้ากราฟ

club_id created_at friend_score
interest_name interest_score name room
score student_id year

	student	club	score
1	"Alice"	"Robotics Club"	8
2	"Alice"	"Photography Club"	2




```
MATCH (me:Student {student_id:'S001'})-[:FRIEND_OF]-(friend:Student)-[:MEMBER_OF]-
>(club:Club)
WHERE NOT (me)-[:MEMBER_OF]->(club)
WITH me, club, count(DISTINCT friend) AS friend_score
OPTIONAL MATCH (me)-[:INTERESTED_IN]->(i:Interest)<-[:FOCUSES_ON]-(club)
WITH me, club, friend_score, count(DISTINCT i) AS interest_score
MERGE (me)-[r:RECOMMENDED]->(club)
SET r.friend_score   = friend_score,
    r.interest_score = interest_score,
    r.score          = friend_score * 2 + interest_score,
    r.created_at     = datetime()
RETURN me.name AS student, club.name AS club, r.score AS score
ORDER BY score DESC;
```

แทนที่จะแสดงผลแล้วจบไป เรานำคะแนนที่คำนวณได้เขียนกลับเข้าไปในกราฟเป็น Relationship ชนิดใหม่ ชื่อ RECOMMENDED พร้อม property เก็บคะแนนและเวลาที่สร้าง ข้อดีคือครั้งต่อไปสามารถดึงคำแนะนำได้ทันทีโดยไม่ต้องคำนวณใหม่ และยังมองเห็นเป็นกราฟได้ด้วยตา เหตุผลที่ใช้ MERGE แทน CREATE คือกันไม่ให้เกิดเส้นเชื่อมซ้ำเมื่อรันคำสั่งเดิมหลายครั้ง

คำอธิบายภาษาไทย

ระบบแนะนำ Robotics Club (C01) เป็นอันดับ 1 ให้ Alice (S001) เพราะเมื่อเดินเส้นทาง (S001)-[:FRIEND_OF]-(friend)-[:MEMBER_OF]->(club) พบว่าเพื่อนของ Alice ถึง 3 คน คือ Bob, Charlie และ Diana เป็นสมาชิกชมรมนี้ ซึ่งเป็นค่า count(DISTINCT friend) ที่สูงที่สุดในกราฟ สะท้อนแรงดึงดูดทางสังคมว่าถ้าเข้าชมรมนี้จะมีคนรู้จักรออยู่แล้ว

นอกจากนี้ยังมีเส้นทางที่สองมาเสริม คือ (S001)-[:INTERESTED_IN]->(i)<-[:FOCUSES_ON]-(C01) ซึ่งตรงกันถึง 2 หัวข้อ (Programming และ AI) หมายความว่าชมรมนี้ไม่ได้ดีแค่เรื่องเพื่อน แต่ตรงกับความสนใจส่วนตัวด้วย คะแนนรวมจึงเป็น 3×2 + 2 = 8

ส่วน Music Club (C02) แม้มีเพื่อนอยู่ แต่ถูกตัดทิ้งเพราะ Alice เป็นสมาชิกอยู่แล้ว และ Photography Club (C03) ได้เพียง 2 คะแนน เพราะมีเพื่อนแค่ 1 คนและไม่มี Interest ที่ตรงกันเลย — นี่คือจุดแข็งของ Graph Database ที่ตอบได้ว่า "ทำไม" ผ่านเส้นทางความสัมพันธ์ ไม่ใช่แค่ตัวเลขคะแนนลอย ๆ